POSTECH · FIAP

DevOps e Arquitetura Cloud

ToggleMaster

Tech Challenge — Fase 01

Cultura DevOps, Arquitetura de Aplicações e Arquitetura Cloud na AWS

Participantes – Grupo 51

Chrystian Pinheiro Farias – RM378246
Lucas Trombini Batista dos Santos – RM378266
Luciano Silva – RM378900
Luiz Guilherme Nunes Fernandes – RM377180
Pedro Lucas Marinho Soares Souza – RM378698

Outubro de 2026 (Fase 1)

Links:

Video: https://www.youtube.com/watch?v=xE9p_n9Ksns

AWS Calculator: https://calculator.aws/#/estimate?id=05590ee9be06a50fa2670879230217781fa1ecc2

Draw.io : Topologia da Arquitetura AWS

GitHub: https://github.com/LucasTrombiniBatista/ToggleMaster-Tech-Challenge.git

1. Introdução

O ToggleMaster é uma plataforma de Feature Flags da DevOps Solutions Inc.

Esta Fase 1 entrega o MVP monolítico (API REST em Python/Flask com PostgreSQL) e a arquitetura para hospedá-lo na AWS e baixo custo.

1.1 Escopo deste documento

- Análise do monolito toggle-master-monolith.
- Vantagens e desvantagens do monolito e análise dos 12 Fatores.
- Arquitetura AWS: diagrama, Security Groups e estimativa de custo.
- Deploy manual na EC2 com RDS (comandos AWS CLI).

2. Cultura DevOps e Arquitetura de Aplicações

2.1 Visão geral da aplicação

O repositório toggle-master-monolith é pequeno e contém apenas seis arquivos:

Arquivo	Função
app.py	Toda a aplicação: configuração, conexão com o banco, criação da tabela e rotas HTTP (Flask).
requirements.txt	Dependências Python: Flask 2.2.2, Werkzeug 2.3.8, psycopg2-binary 2.9.5 e gunicorn 20.1.0.
Dockerfile	Imagem baseada em python:3.9-slim; instala dependências e o postgresql-client; executa o entrypoint.
entrypoint.sh	Valida variáveis de ambiente, aguarda o PostgreSQL (pg_isready), roda flask init-db e inicia o Gunicorn.
docker-compose.yaml	Sobe a aplicação (porta 5000) e um PostgreSQL 13 com volume persistente e healthcheck.
README.md	Instruções de execução local, deploy na EC2 e entregáveis.

A API expõe os seguintes endpoints:

Método	Endpoint	Descrição	Retornos
GET	/health	Health check da aplicação.	200
POST	/flags	Cria uma flag (campo name obrigatório; is_enabled opcional, padrão false).	201, 400, 409
GET	/flags	Lista todas as flags ordenadas por nome.	200
GET	/flags/<name>	Retorna o status de uma flag.	200, 404
PUT	/flags/<name>	Atualiza is_enabled (booleano obrigatório).	200, 400, 404

Os dados ficam em uma única tabela, criada automaticamente pelo comando flask init-db:

```
CREATE TABLE IF NOT EXISTS flags (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    is_enabled BOOLEAN NOT NULL DEFAULT false,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
);
```

2.2 Execução local da aplicação

Pré-requisitos: Docker e Docker Compose instalados, além do curl (ou Postman/Insomnia) para testar a API.

1. Extrair o projeto e entrar no diretório:

```
unzip toggle-master-monolith-main.zip
cd toggle-master-monolith-main
```

2. Construir a imagem e subir os contêineres (aplicação + PostgreSQL):

```
docker compose up --build
```

3. Em outro terminal, validar o health check:

```
curl http://localhost:5000/health
# resposta esperada: {"status": "ok"}
```

4. Criar, listar, consultar e atualizar uma flag:

```
curl -X POST -H "Content-Type: application/json" \
     -d '{"name": "new-feature", "is_enabled": true}' \
     http://localhost:5000/flags

curl http://localhost:5000/flags
curl http://localhost:5000/flags/new-feature

curl -X PUT -H "Content-Type: application/json" \
     -d '{"is_enabled": false}' \
     http://localhost:5000/flags/new-feature
```

Como a execução local funciona

Com compose, o serviço db executa o healthcheck. A aplicação só inicia depois que o banco está saudável. Dentro do contêiner, o entrypoint.sh confirma que DB_HOST, DB_PORT e DB_NAME existem, aguarda o banco aceitar conexões, executa flask init-db para criar a tabela e finaliza com exec gunicorn.

2.3 Por que a aplicação é um monolito?

Um monolito é uma aplicação construída, implantada e escalada como uma única unidade. No ToggleMaster isso se evidencia em vários pontos:

Característica	Evidência no código
Uma única unidade de deploy	Um só processo (gunicorn app:app) e uma só imagem Docker. Qualquer alteração exige novo build e novo deploy de tudo.
Todas as responsabilidades no mesmo código	Em app.py convivem configuração, conexão com o banco, DDL (init_db), validação de entrada, regras e rotas HTTP. Não há separação em camadas ou módulos.
Banco de dados único e compartilhado	Uma única base PostgreSQL com a tabela flags, acessada diretamente por todos os handlers.

Característica	Evidência no código
Acoplamento forte	O SQL está escrito dentro de cada rota. Trocar o banco ou separar um domínio exigiria alterar vários pontos.
Escala conjunta	Não é possível escalar só a leitura de flags ou só a escrita: a aplicação inteira é replicada ou nada é.
Falha compartilhada	Se o processo ou a conexão com o banco falhar, todos os endpoints ficam indisponíveis.

2.4 Vantagens e desvantagens para um MVP

Vantagens (por que faz sentido agora)	Desvantagens (limites a considerar)
Simplicidade: um repositório, um processo, um deploy.	Escalabilidade limitada: escala-se tudo junto, mesmo que só um ponto seja gargalo.
Velocidade de entrega: colocar no ar e receber feedback rapidamente é o objetivo do MVP.	Ponto único de falha: uma só EC2 e um banco Single-AZ; uma falha derruba o serviço.
Custo baixo: uma instância pequena e um banco pequeno.	Deploys com risco: qualquer mudança reinicia a aplicação inteira, podendo gerar indisponibilidade.
Operação e depuração fáceis: um único log, um único lugar para investigar.	Acoplamento crescente: com o aumento de funcionalidades o código tende a ficar difícil de manter.
Sem latência de rede entre serviços e sem a complexidade de sistemas distribuídos.	Dificulta o trabalho de várias equipes em paralelo e a adoção de tecnologias diferentes por domínio.
Transações e consistência simples, com um único banco.	Exige refatoração futura para migrar para microsserviços (previsto nas próximas fases).

2.5 Análise dos 12 Fatores (12-Factor App)

A tabela a seguir avalia cada fator com base no código atual e indica os ajustes recomendados para um ambiente de produção mais robusto.

#	Fator	Situação	Evidência no projeto	Ajuste para produção
I	Base de código	Atende	Um único repositório versionado no Git, com um único deploy.	Manter. Futuramente, um repositório por serviço quando houver microsserviços.
II	Dependências	Parcial	requirements.txt com versões fixas (pinned) e Dockerfile que as instala de forma isolada.	Atualizar o Python e as bibliotecas; adicionar varredura de vulnerabilidades. Declarar explicitamente o click, hoje herdado do Flask.
III	Configuração	Parcial	Credenciais e endpoint lidos de variáveis de ambiente (DB_HOST, DB_NAME, DB_USER, DB_PASSWORD), sem segredos no código. O compose traz senha fixa, apenas para uso local.	A aplicação não lê DB_PORT (usa o padrão 5432) e não valida as variáveis na inicialização. Em produção, usar AWS Secrets Manager ou SSM Parameter Store em vez de export manual.
IV	Serviços de apoio	Atende	O PostgreSQL é um recurso externo anexado por DB_HOST: local (contêiner) ou RDS, sem alterar o código.	Manter. Considerar pool de conexões e tempos limite (timeouts) de conexão.

#	Fator	Situação	Evidência no projeto	Ajuste para produção
V	Build, release, run	Parcial	O Dockerfile separa o build da execução, mas o processo é manual. O compose monta o código com volumes (.:/app), apenas para desenvolvimento.	Criar pipeline de CI/CD, imagens com tag de versão e releases imutáveis. Remover o volume de código fora do desenvolvimento.
VI	Processos	Atende	A aplicação é stateless: nenhum estado em memória ou disco, tudo persiste no PostgreSQL.	Manter. Isso permite escalar horizontalmente no futuro.
VII	Vínculo de porta	Atende	O Gunicorn expõe o serviço por conta própria na porta 5000 (0.0.0.0:5000), sem depender de um servidor web externo.	Manter. Em produção, colocar um proxy reverso/ALB na frente.
VIII	Concorrência	Parcial	O Gunicorn é executado com a configuração padrão (um único worker).	Definir o número de workers conforme as CPUs e escalar por mais instâncias do processo (ASG/contêineres).
IX	Descartabilidade	Parcial	Inicialização rápida, espera pelo banco e uso de exec no entrypoint (sinais chegam ao Gunicorn). Porém, flask init-db roda a cada start com || true, escondendo falhas.	Tratar o desligamento (graceful shutdown) e separar a criação do schema da inicialização. Usar health checks reais (readiness/liveness).
X	Paridade dev/prod	Parcial	Mesmo motor (PostgreSQL) e mesmo código em todos os ambientes. Local usa Postgres 13; a versão do RDS pode divergir.	Alinhar a versão do PostgreSQL local com a do RDS e usar a mesma imagem/artefato em todos os ambientes.
XI	Logs	Parcial	Saída em stdout/stderr com PYTHONUNBUFFERED=1, como o fator recomenda. Mensagens com print() e sem estrutura.	Adotar logging estruturado (JSON), habilitar o access log do Gunicorn e enviar a um agregador (CloudWatch Logs). Não devolver detalhes de exceções (str(e)) ao cliente.
XII	Processos administrativos	Parcial	Existe o comando administrativo flask init-db, executável de forma pontual.	Usar uma ferramenta de migração de schema (ex.: Alembic/Flyway) e executar tarefas administrativas como jobs separados.

Resumo: a aplicação já atende 4 dos 12 fatores (I, IV, VI e VII) e atende parcialmente os outros 8, com pontos de melhoria, sobretudo em configuração e segredos, logs, descartabilidade e automação de build/release.

3. Arquitetura Cloud

A arquitetura proposta é deliberadamente enxuta, evitando serviços que geram custo fixo mensal.

3.1 Diagrama de arquitetura

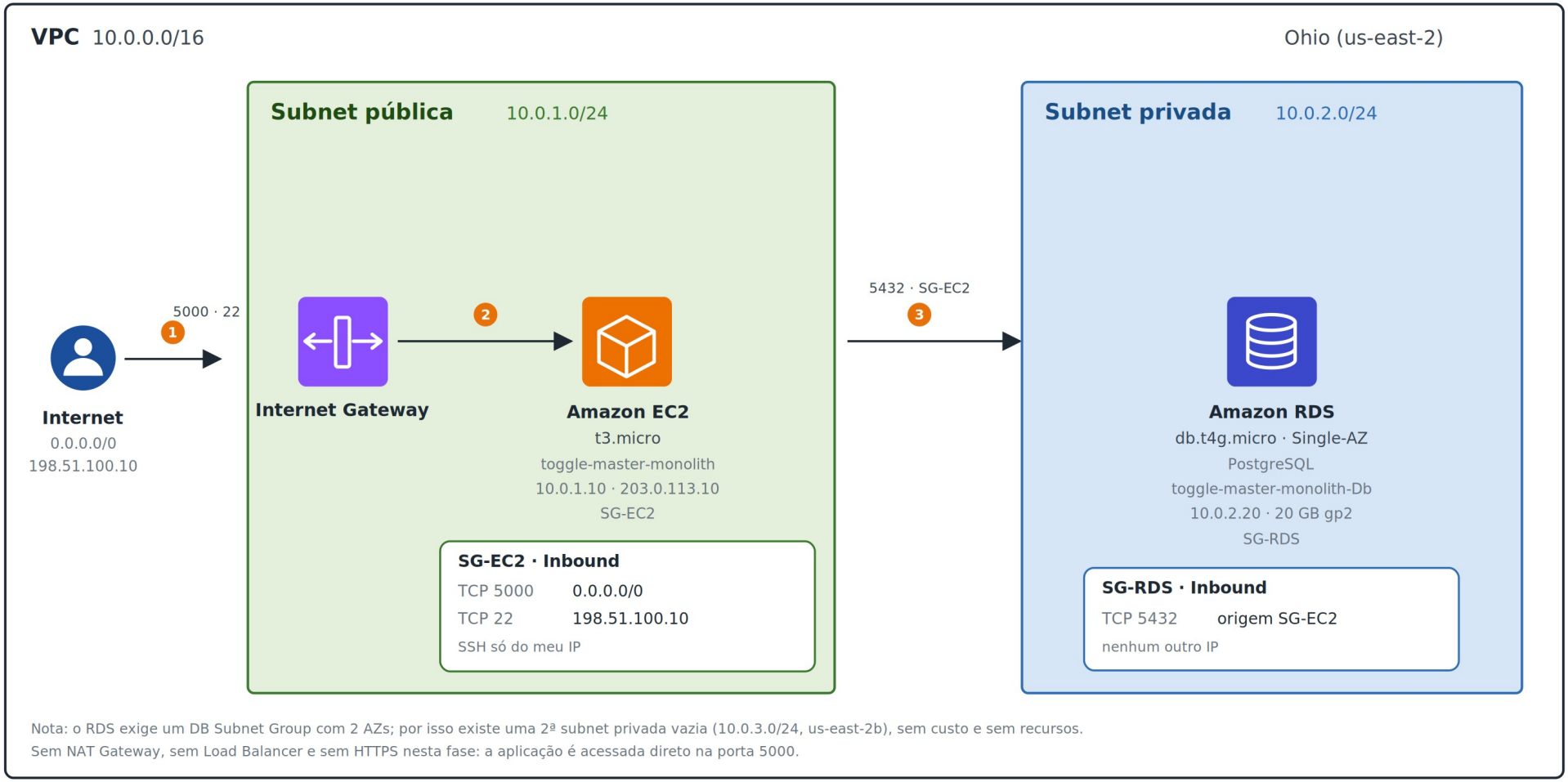


Figura 1 — Arquitetura AWS do ToggleMaster (porta 5000 publicada; RDS acessível somente pelo SG-EC2).

3.2 Componentes e justificativas

Componente	Configuração proposta	Justificativa
VPC	10.0.0.0/16 na região us-east-2	Rede isolada e privada da aplicação. Não gera custo.
Sub-rede pública	10.0.1.0/24 (us-east-2a), com rota 0.0.0.0/0 para o Internet Gateway	Abriga a EC2, que precisa ser acessível pela internet.
Sub-redes privadas	10.0.2.0/24 (us-east-2a) e 10.0.3.0/24 (us-east-2b), sem rota para a internet	Isolam o banco. O RDS exige um DB Subnet Group com sub-redes em pelo menos duas AZs; a segunda fica vazia e não gera custo.
Internet Gateway	Anexado à VPC	Permite a entrada e saída de tráfego da sub-rede pública. Não gera custo.
EC2	t3.micro, Amazon Linux 2023, disco gp3 de 20 GB, IP público, acesso por chave .pem	Menor instância adequada para um MVP com Flask/Gunicorn. Executa a aplicação na porta 5000.
RDS PostgreSQL	db.t4g.micro, Single-AZ, 20 GB, sem acesso público, em sub-rede privada	O banco fica inalcançável pela internet. Single-AZ reduz o custo (Multi-AZ dobraria o preço).
Security Groups	SG-EC2 e SG-RDS (detalhados a seguir)	Firewall com estado, aplicando o princípio do menor privilégio.

3.3 Security Groups

SG-EC2 (instância da aplicação)

Direção	Protocolo / Porta	Origem / Destino	Finalidade
Entrada	TCP 5000	0.0.0.0/0	Porta publicada: acesso direto à API (Gunicorn).
Entrada	TCP 22 (SSH)	198.51.100.10/32	SSH somente do meu IP (nunca 0.0.0.0/0).
Saída	Todo o tráfego	0.0.0.0/0	Padrão: instalação de pacotes e conexão com o RDS.

SG-RDS (banco de dados)

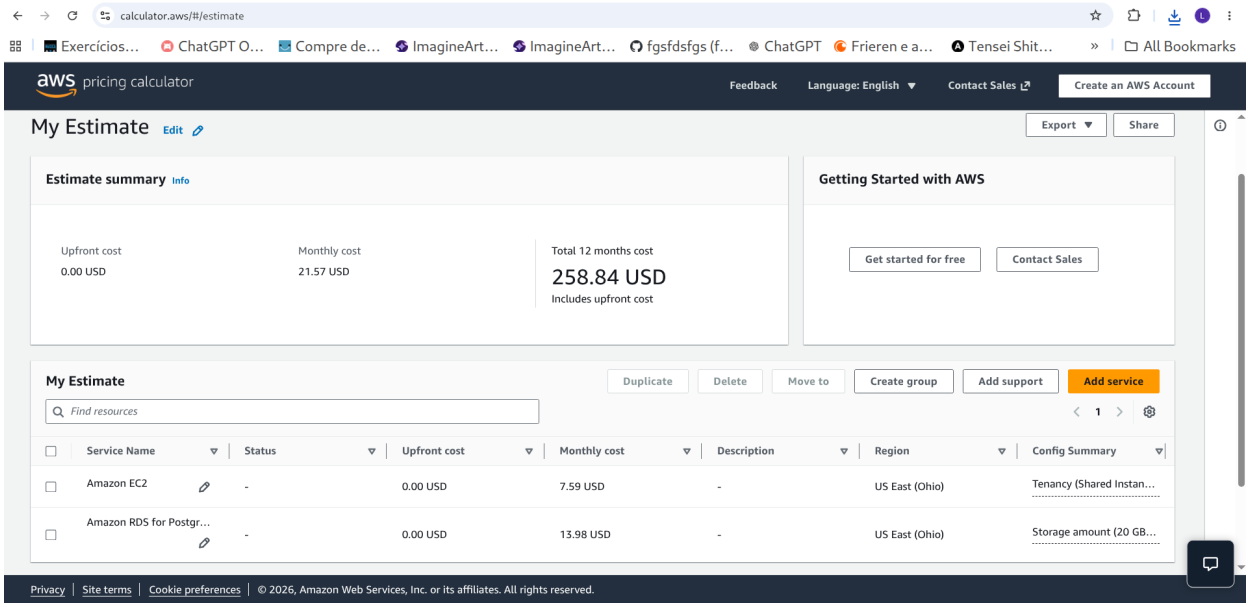
Direção	Protocolo / Porta	Origem / Destino	Finalidade
Entrada	TCP 5432 (PostgreSQL)	SG-EC2	Somente a EC2 acessa o banco; nenhum outro IP.
Saída	Padrão	—	Sem regras adicionais.

A porta 5000 fica publicada para a internet e é atendida diretamente pelo Gunicorn, sem Nginx, proxy ou Load Balancer a conexão não é criptografa. Sugestão: para a conexão **HTTPS,** o tráfego fica criptografado, mas o certificado não é validado por uma autoridade confiável, então a solução serve para teste e MVP, e os clientes precisam aceitar o aviso ou usar `curl -k`. O acesso ao banco é restrito: o RDS aceita a porta 5432 somente com origem no SG-EC2, sem nenhum outro IP e sem acesso público. O SSH (porta 22) é liberado apenas para o meu IP, nunca para 0.0.0.0/0.

3.4 Decisões para reduzir custos

- **Sem NAT Gateway:** a EC2 fica em sub-rede pública com IP público, e o RDS, em sub-rede privada, não precisa de saída para a internet. Um NAT Gateway custaria algo em torno de US$ 33 por mês só pela hora de uso.
- **Sem Load Balancer:** a aplicação é acessada direto pela EC2. Um ALB acrescentaria cerca de US$ 16 por mês, e fica como evolução para as próximas fases.
- **RDS Single-AZ:** um único banco em uma AZ; alta disponibilidade (Multi-AZ) fica fora do escopo do MVP.
- **Menores instâncias possíveis:** t3.micro na EC2 e db.t4g.micro no RDS, com 20 GB de armazenamento cada, o suficiente para o MVP.
- **Recursos de rede gratuitos:** VPC, sub-redes, Internet Gateway, route tables e Security Groups não têm custo.
- **Boa prática:** desligar/parar a EC2 e o RDS ao fim dos testes e remover os recursos após a avaliação.

3.5 Estimativa de custo mensal



3.6 Deploy manual na AWS

A infraestrutura do diagrama foi provisionada manualmente no console da AWS. Segue o resumo próximo em AWS CLI e o deploy na EC2.

1. Rede e Security Groups

```
export AWS_REGION=us-east-2 MEUIP=198.51.100.10/32
VPC=$(aws ec2 create-vpc --cidr-block 10.0.0.0/16 --query Vpc.VpcId --output text)
PUB=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.0.1.0/24 --availability-
zone us-east-2a --query Subnet.SubnetId --output text)
PRV1=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.0.2.0/24 --availability-
zone us-east-2a --query Subnet.SubnetId --output text)
PRV2=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.0.3.0/24 --availability-
zone us-east-2b --query Subnet.SubnetId --output text)
aws ec2 modify-subnet-attribute --subnet-id $PUB --map-public-ip-on-launch
IGW=$(aws ec2 create-internet-gateway --query InternetGateway.InternetGatewayId --
output text)
aws ec2 attach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC
```

```
RT=$(aws ec2 create-route-table --vpc-id $VPC --query RouteTable.RouteTableId --
output text)
aws ec2 create-route --route-table-id $RT --destination-cidr-block 0.0.0.0/0 --
gateway-id $IGW
aws ec2 associate-route-table --route-table-id $RT --subnet-id $PUB

SGEC2=$(aws ec2 create-security-group --group-name SG-EC2 --description
ToggleMaster-EC2 --vpc-id $VPC --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $SGEC2 --protocol tcp --port
5000 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id $SGEC2 --protocol tcp --port 22
--cidr $MEUIP
SGRDS=$(aws ec2 create-security-group --group-name SG-RDS --description
ToggleMaster-RDS --vpc-id $VPC --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $SGRDS --protocol tcp --port
5432 --source-group $SGEC2
```

2. RDS PostgreSQL (privado)

```
aws rds create-db-subnet-group --db-subnet-group-name togglemaster-subnets \
  --db-subnet-group-description ToggleMaster --subnet-ids $PRV1 $PRV2
aws rds create-db-instance --db-instance-identifier toggle-master-monolith-db \
  --engine postgres --db-instance-class db.t4g.micro --allocated-storage 20 --
storage-type gp2 \
  --db-name togglemaster --master-username <usuario> --master-user-password
'<senha>' \
  --db-subnet-group-name togglemaster-subnets --vpc-security-group-ids $SGRDS \
  --no-publicly-accessible --no-multi-az
```

3. EC2 (Amazon Linux 2023, t3.micro)

```
AMI=$(aws ssm get-parameter --name /aws/service/ami-amazon-linux-latest/al2023-ami-
kernel-default-x86_64 --query Parameter.Value --output text)
aws ec2 create-key-pair --key-name togglemaster-key --query KeyMaterial --output
text > togglemaster-key.pem && chmod 400 togglemaster-key.pem
aws ec2 run-instances --image-id $AMI --instance-type t3.micro --key-name
togglemaster-key \
  --subnet-id $PUB --security-group-ids $SGEC2 --associate-public-ip-address \
  --block-device-mappings 'DeviceName=/dev/xvda,Ebs={VolumeSize=20,VolumeType=gp3}'
\
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=toggle-master-
monolith}]'
```

4. Acesso SSH, instalação e execução

```
ssh -i togglemaster-key.pem ec2-user@<IP_PUBLICO_EC2>
sudo dnf update -y && sudo dnf install -y git python3 python3-pip
git clone <url-do-repositorio> && cd toggle-master-monolith
python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt
export DB_HOST='<endpoint-do-rds>' DB_NAME='togglemaster' DB_USER='<usuario>'
DB_PASSWORD='<senha>'
flask init-db                                  # cria a tabela flags no RDS
gunicorn --bind 0.0.0.0:5000 app:app
```

5. Validação

```
curl http://<IP_PUBLICO_EC2>:5000/health
curl -X POST -H 'Content-Type: application/json' -d '{"name":"new-
feature","is_enabled":true}' http://<IP_PUBLICO_EC2>:5000/flags
curl http://<IP_PUBLICO_EC2>:5000/flags
```

As credenciais ficam apenas em variáveis de ambiente (nunca no Git). A API responde pelo IP público na porta 5000, e o RDS só é alcançável pela EC2 via SG-EC2.

3.7 Desafio

Este projeto permitiu compreender, na prática, como um MVP monolítico pode ser colocado no ar na AWS com simplicidade e baixo custo. Os principais desafios foram avaliar a aplicação frente aos 12 Fatores, desenhar uma arquitetura enxuta (sem NAT Gateway, Load Balancer ou Multi-AZ) e proteger o ambiente, com a porta 5000 publicada, o SSH liberado apenas para o nosso IP e o RDS acessível somente pelo Security Group da EC2. Também foi preciso atender à exigência do RDS por duas zonas de disponibilidade e escolher entre HTTP e HTTPS com certificado autoassinado, equilibrando segurança e custo

3.8 Referências

- Repositório do desafio: github.com/dougls/toggle-master-monolith
- The Twelve-Factor App: 12factor.net
- AWS Pricing Calculator: calculator.aws
- Enunciado do Tech Challenge Fase 01 — POSTECH FIAP